



Monte carlo tree search for dynamic bike repositioning in bike-sharing systems

Jianbin Huang¹ · Qinglin Tan¹ · He Li¹ · Ao Li¹ · Longji Huang¹

Accepted: 31 May 2021

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract

With the popularity of green travel and the aggravation of traffic congestion, Bike Sharing System (BSS) is adopted increasingly in many countries nowadays. However, the BSS is prone to be unbalanced because of the unequal supply and demand in each station, which leads to the loss in customer requirements. To address this issue, we develop a Monte Carlo tree search based Dynamic Repositioning (MCDR) method, which can help operators to decide at any time: (i) which station should be balanced firstly, and (ii) how many bikes should be picked or dropped at an unbalanced station. In this paper, we first employed a Density-based Station Clustering algorithm to reduce the problem complexity. Then the concept of service level is introduced to calculate the number of bikes that need to be transferred at each station. Finally, considering multiple factors, we propose a dynamic bike repositioning approach named MCDR, which can provide an optimal repositioning strategy for operators. Experimental results on a real-world dataset demonstrate that our method can reduce customer loss more effectively than the state-of-the-art methods.

Keywords Bike sharing system · Spatio-temporal data analysis · Monte carlo tree search · Dynamic bike repositioning

1 Introduction

The bike-sharing system, convenient and environmentally friendly, emerged in the wake of the growing demand for green travel. According to [1], there are 1700 BSSs already in use and 390 systems under the planning or preparation phase. Commonly, a station can hold only a limited number of bikes. If a customer rents or returns a bike at the station, a bike usage record will be generated. According to the collected

data, we can obtain real-time information about available bikes or free docks at each station, reflects the relationship between supply and demand. Since the BSS allows users to rent a bike at one station and return it at another station [2], the Spatio-temporal distribution of the bikes is unbalanced, as shown in Fig. 1. When nearby stations are not available, customers have to find another station to rent or return a bike. It may eventually hinder customers from participating in BSS, resulting in huge customer losses. Although operators can use trucks to bring constant balance among stations, there still facing many difficulties and challenges in figuring out a good bike repositioning strategy.

Large volume of BSS Generally, a BSS is composed of hundreds of stations and covers a large district. Direct relocation of bikes on such a large system may result in irregular distribution, missing unbalanced stations, or overlapping relocation of multiple trucks. In addition, due to cost and efficiency factors, it is not appropriate to guide trucks to transfer bicycles throughout the region.

Impact of multiple factors Many factors in BSS influence the decision of the next station that needs to be balanced most. Purely for simplicity, we only consider the main four factors list as follows: 1) the distance between stations and

✉ Qinglin Tan
qltan@stu.xidian.edu.cn

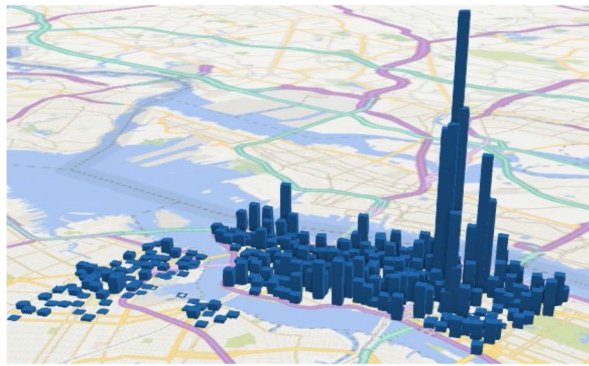
Jianbin Huang
jbhuang@xidian.edu.cn

He Li
heli@xidian.edu.cn

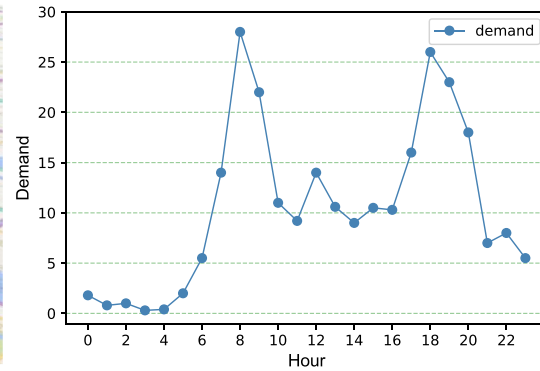
Ao Li
aoli626@163.com

Longji Huang
longji.huang018@gmail.com

¹ School of Computer Science and Technology, Xidian University, Xi'an, 710071, China



(a) Spatial distribution



(b) Temporal distribution

Fig. 1 Spatio-temporal distribution of bike usage pattern

trucks. As an important factor in truck scheduling problem, distance is closely related to scheduling cost and efficiency. 2) the state of stations. In fact, most stations are still balanced and unnecessary for trucks to stay, e.g., s_2 and s_6 in Fig. 2. 3) the subsequent unbalanced stations. A balanced station may surround by a lot of unbalanced stations like s_6 in Fig. 2, guide trucks to such stations can greatly improve scheduling efficiency. 4) the number of bikes transferred (TN). As shown in Fig. 2, a positive TN value represents the number of bikes that need to be picked up at that unbalanced station, and the negative one represents the number that needs to be put down. This factor reflects the urgency of the stations and can help us prioritize stations. It is challenging to combining all of them into the development of the optimal repositioning strategy.

Huge solution space and dynamic Same as the route recommendation problem, the optimal strategy selection problem also has a large solution space. **There are many possible choices of the next station for a truck to move to, and it is challenging to find the best one among them.** Moreover, bike-use requests come stochastically and may be influenced by environmental factors like weather and time of day, making the BSS dynamic.

To overcome these challenges outlined above and fill the gaps in existing works, we proposed a Monte Carlo-based dynamic repositioning algorithm named MCDR for bike repositioning in the BSS. By considering multiple factors simultaneously and forecasting the demand of each station in advance, our method can adapt to the complex environment of the BSS. The main contributions of this paper are summarized as follows:

1. We employed a density-based station clustering method to divide the region into multiple clusters so as decouple the problem to reduce computational complexity.
2. By considering a variety of external factors, we design two demand predictors to predict the future rental and return demand and calculate the service level of each site. Based on the service level, we can find out

the unbalanced stations and obtain the corresponding number of bikes that need to be transferred.

3. A Monte Carlo tree search based Dynamic Repositioning (MCDR) method is proposed to guide each truck which station should go to at the next period. We also designed a novel reward value Q to determine the priority of each station to guarantee the efficacy of the repositioning process.
4. We evaluate our method on a real-world dataset from the New York City Bike system. Experimental results validate the efficacy of the proposed solution methodology over the state-of-the-art approaches.

The rest of this paper is organized as follows. Section 2 gives a brief presentation of some recent related work. In Section 3, we introduce some definition and present the framework of our method. In Section 4, we propose our methods and illustrate each module in detail. Then the experimental results and the performance of our algorithm are shown in Section 5. Finally, we discuss our conclusion and future work in Section 6.

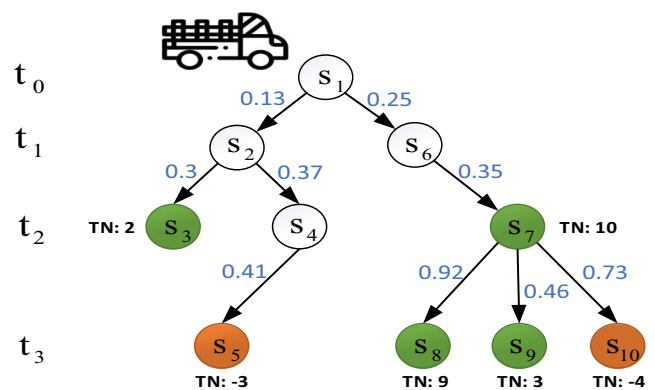


Fig. 2 An example for bike repositioning. Each circle denotes a station in the BSS, and those circles with red/green color mean those stations will be stuffed/starved in the future. The number associated with each edge represents the distance between the two connected nodes (stations), in kilometers

2 Related work

2.1 Bike demand prediction

Station imbalance in bike-sharing systems has been one of the main focuses for many years. Since predicting the bike demand is conducive to rebalance the bikes of the bike-sharing systems, many researchers have been devoted to this field. For instance, Hulot et al. [3] uses the temporal and weather features to predict the hourly demand of each station. Sathishkumar et al. [4] introduced data mining techniques to further utilize the historical patterns of bikes in the station. Instead of predicting bike demand, Cagliero et al. [5] designed a data mining framework called STation Occupancy Predictor (STOP) to forecast the occupancy levels of stations. This model based on the current and history station occupancy values is able to be used for characterizing system usage as well. In [6], Zhou et al. proposed a novel method to predict the traffic flow between stations by exploiting a clustering algorithm and the temporal link prediction strategy. Besides doing a forecast for each station independently, some studies focus more on the larger scale prediction. Jia et al. [7] first cluster stations of the system into groups, and then presented a hierarchical prediction model to deduce the take/deposit of bikes to each cluster in the next period. In [8], Li et al. also employed a clustering algorithm to divide bike stations into smaller groups, but they adopted a Transition based Inference (TINF) method to predict citywide bike usage in the near future. Based on these studies, we designed two demand predictor to infer the bike usage and applied a density-based clustering method to reduce the computational complexity and improve efficiency.

2.2 Bike reposition

In the context of the bike-sharing system, modeling how to rebalance the bikes between stations in an efficient way is of importance for the operators to manage the whole bikes-sharing system. The bike reposition problem has been extensively studied in recent years. We divided works about this topic into three categories: static repositioning, dynamic repositioning, and user-based repositioning.

Static repositioning Static bike repositioning does not consider the dynamic flow of bikes between stations, and they assume the redistribution of bikes only occurs at the beginning or end of each day. Previous work in [9, 10] solving static repositioning relies on optimization model, which usually based on some objectives, such as minimizing the total distance traveled, maximizing revenue for the BSS company, or minimizing the total CO_2 emissions of trucks. In [11], Wang et al. studied a more realistic

problem. They considered the relocation of good and bad bikes simultaneously and proposed a Mixed Integer Linear Programming method to solve this problem economically. Vogel et al. [12] first utilize a data mining method to analyze the activity patterns of bikes and then make an appropriate location planning for stations according to these patterns. This kind of approach can not respond to the change of bicycle distribution in the system in time, which reduces the efficiency of the system and even cause a system outage.

Dynamic repositioning Dynamic repositioning is the problem of generating planning by considering the real-time usage of the bikes. Ghosh et al. [13, 14] proposed an approach based on the Lagrangian Dual Decomposition and an abstraction mechanism to address the dynamic redeployment problem of bikes and the routing problem for carriers simultaneously. Lowalekar et al. [15] developed a multi-stage stochastic optimization formulation to relocate and routing for the trucks in the BSS. Assuming that the environment may confront the current relocation solution by generating worse demand scenarios, Ghosh et al. [16] employed a scenario generation based two-player iterative game to computing an available strategy. And in [17], they designed an optimization model to maximize the probability of satisfying the bike demand. Although the dynamics are considered in this model, they can still not change repositioning strategies in real-time to suit the new environment. We also noticed that most of those methods try to determine the optimal number of transferred bikes without considering the factors that influence the route selection of repositioning bikes. Qin et al. [18] first design the Spatial-Temporal Bike Flow Prediction (ST-BFP) model, which is a convolutional network based on a residual framework. Then they proposed a local search algorithm to minimize the travel time of carriers between regions in the dockless bike-sharing system. A multi-objective optimization model is designed in [19] based on a modified artificial bee colony algorithm. Li et al. [20] maximized the long-term reward of a decision sequence in a reinforcement learning way, but they ignored some useful factors (e.g. the travel distance). Since the reward is only related to customer loss, it may cause long-distance traveled in low-demand situations.

User-based repositioning In user-based bike repositioning, operators usually incentivize the customers to rent or return a bike at specific stations with a reward. Pfrommer et al. [21] and Singla et al. [22] try to encourage customers to participate in the bicycle relocation process by using the dynamic pricing strategy with truck assistance. Neither employs a fleet of carrier vehicles nor incentives users for bike repositioning tasks. Ghosh et al. [23] proposed to relocate bikes dynamically by the bike trailers and designed a crowdsourcing mechanism to assign

the repositioning tasks to the potential users. In [24], Angelopoulos et al. adopted a graph-theoretic approach to model the vehicle imbalance problem in the vehicle sharing systems (VSS) and presented two reward schemes to incent users to participate in the vehicle rebalance process. Before assigning stations to customers, Yi et al. [25] employed a one-dimensional random walk process with two absorption walls to calculate the optimal state of each station, and then recommends the most suitable station for each user. With the belief of improving user satisfaction, Ji et al. [26] recommend stations for users from the perspective of user utility maximization. However, these user-based methods weaken the management role of operators, and the direct intention of users limited the efficiency of relocation. Chiarioti et al. proposed a joint optimization model in [27] to improve system service quality and reduce scheduling costs by combining vehicle dispatch and user incentive measures. However, how to assign suitable rewards to customers in different situations remains a challenge. Furthermore, the effectiveness of these models depends on the participation of users, which can not guarantee the continuity of the relocation process.

Recently, the **MCTS** model shows tremendous potential in **taxi route recommendation** [28], but due to the **extreme** difference between taxi and BSS, this model cannot be used in bike repositioning problem directly. Different from those models mentioned above, we designed a novel **MCTS-based model named MCDR**. This model not only considers the dynamics of BSS but also considers the influence of many factors on bicycle relocation. Moreover, we provide flexible options in different demand situations.

3 Overview

Some notations used in this paper are defined first in Table 1, then we give an overview of our framework. In order to capture the demand patterns at different times, we divided the time periods in one day into three partitions: morning rush hours p_1 (7:00 - 10:00), day time p_2 (11:00 - 4:00) and evening rush hours p_3 (17:00 - 20:00).

3.1 Preliminary

Definition 1 (Customer Loss) If a customer failed to rent/return a bike at an empty/full station, we defined this event as a customer loss. Therefore, the customer loss can be computed by subtracting the number of bikes or docks left at the station from the actual demand. In our MCDR model, we mainly focus on three partitions (i.e. p_1 , p_2 , and p_3) and repositioning bikes in each of them independently. In each

Table 1 Table of Notations

Notation	Description
s_i	The i^{th} station
C_s	The capacity of the station s
$\lambda_s(t)$	The expected demand for returning bikes at time t in station s
$\mu_s(t)$	The expected demand for renting bikes at time t in station s
$D_{s,p_i}^{k,+}/D_{s,p_i}^{k,-}$	The return(+) or rental(-) demand for station s in the period p_i of day k
$x_{s,t}$	The real-time bikes at time t in station s

time-span, we defined the total loss of a station as follow:

$$loss = loss_{br} + loss_{ar}, \quad (1)$$

where $loss_{br}$ represents the customer loss before the truck arrives, $loss_{ar}$ is the customer loss after bike reposition. Note that the $loss_{ar}$ will be 0 if there is no rebalancing demand in the station, and the $loss_{br}$ will be 0 when the station is unbalanced at the beginning of the partition. In order to maintain a unified form of the loss, we use the total customer loss to represent the final loss of each station.

Definition 2 (δ -neighbourhood) The δ -neighbourhood of a station s_i is defined as $N_i^\delta = \{s_j \mid s_j \in S, Dist(s_i, s_j) \leq \delta\}$, which is a set of stations that the distance between s_i and each of them less than the distance threshold δ where $Dist(s_i, s_j)$ denotes the geographic distance between station s_i and s_j .

Definition 3 (Demand Centrality of Station) Given a station s_i and its δ -neighbourhood, the demand centrality of station s_i is formalized as the bike demand ratio of station s_i to its δ -neighbourhood, which can represent as follow:

$$Cent_{s_i} = \frac{|N_i^\delta| * E_{s_i}}{\sum_{s_j \in N_i^\delta} E_{s_j}}, \quad (2)$$

where E_{s_i} is the total bike demand of station s_i in history and N_i^δ represents the δ -neighbourhood of station s_i .

3.2 Problem statement

Given the bike usage data, the history station status data, and the corresponding meteorology data in the BSS. We first divide the system into several groups, then within each group, we predict the actual bike demand of each station and calculate their real-time service level. Based on those intermediate results, we aim to decide the next station and the corresponding number of bikes that need to be transferred to minimize customer loss.

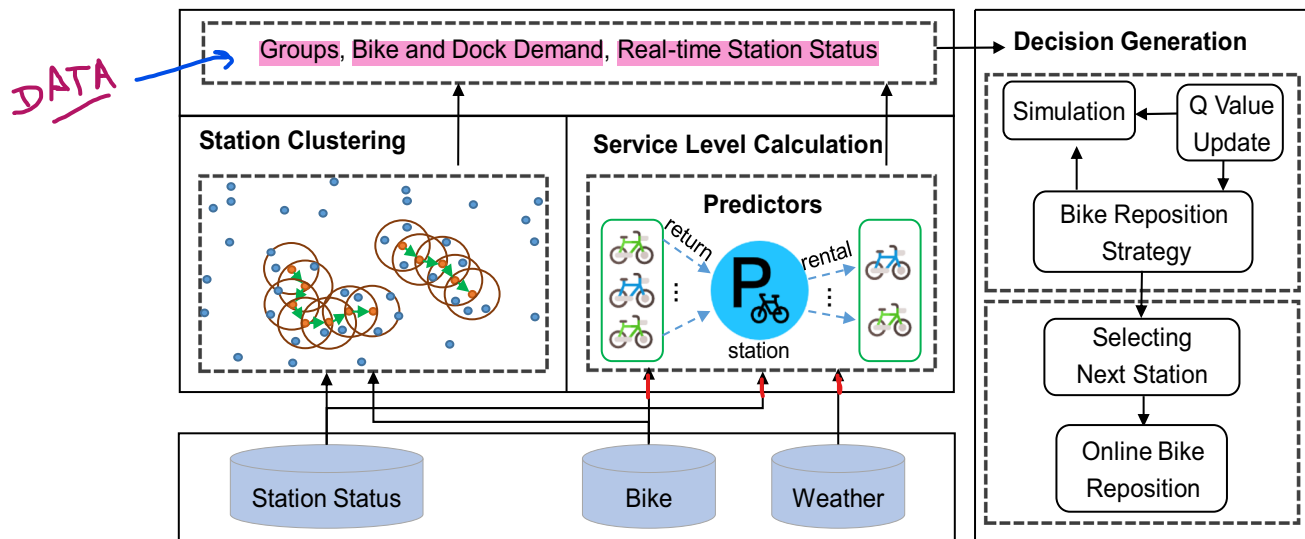


Fig. 3 An overview of the framework

3.3 Framework

As shown in Fig. 3, the method we proposed mainly consists of three components: the stations clustering module, the service level calculation module, and the decision generation module for each cluster. $\times$

the urgency of stations, and the subsequent unbalanced subsequence. After multiple simulations, every station visited by trucks in this process will get a reward value. The station with the maximum reward value is selected as the next target station.

4 Methodology

In this section, we detail the three modules in our method, including the density-based stations clustering module, the service level calculation module, and the decision generation module.

4.1 Stations clustering

In general, the complicity of BSS makes it impractical to reposition the bike directly. Here we introduce a density-based station clustering algorithm to partition the whole BSS into groups according to the similarity of bike usage patterns. Then we can learn the optimal bike reposition strategy in each group instead of the entire BSS.

4.1.1 Central station selection

Central station [30] denote stations with a relatively high proportion of bike demand in a local region, which is detected by a centrality threshold τ . If the bike demand of a station larger than the threshold i.e. $Cent_{s_i} \geq \tau$, we say this station is a central station. The central stations usually close to Points of Interest (POIs) [31] which have large crowd flows such as companies, schools, subways, and parks. The service quality of central stations has a tremendous impact on a local region. Moreover, since a central station is usually

1 The stations clustering module Given that the hugeness and complexity of BSS, reposition bikes in such an enormous system is challenging. Therefore, we employed a density-based clustering method to aggregate those nearby stations with similar bike usage patterns into a group. Then trucks are allocated to each group to conduct intra-cluster reposition among stations.

2 The service level calculation module To determine whether a station will be unbalanced for the coming period, we introduced the concept of service level and designed two predictors (rental and return predictors) to simulate bike flow in the system. Based on the station status data, historical bicycle usage data, and meteorological data, these two predictors can simulate the car rental and return events of each station through the Poisson process [29]. Please note that we get service levels by keeping the status of each station in a position where there are always bikes/docks for customers to rent/return.

3 The decision generation module The purpose of this module is to learn an optimal intra-cluster bike reposition strategy. When a truck needs to select the next unbalanced station at the current location s , the simulated search process will be performed multiple times, which is a learning process. This process involves many factors, including travel distance, the number of bicycles to be transferred,

4 Factors

surrounded by some common stations, we choose them as the core stations in the clustering process.

4.1.2 Distance and bike usage pattern similarity

Before clustering stations into groups, the similarity between stations needs to be measured at first. Based on the bike usage patterns between weekdays, we construct the demand vector of station s like

$$F_s = [D_{s,p_1}^{Mon,+}, D_{s,p_1}^{Mon,-}, D_{s,p_2}^{Mon,+}, D_{s,p_2}^{Mon,-}, \dots, D_{s,p_3}^{Fri,+}, D_{s,p_3}^{Fri,-}]. \quad (3)$$

Given the F_{s_i} of station s_i and F_{s_j} of station s_j , we utilize the Pearson Correlation Coefficient [32] to calculate the similarity of bike usage patterns between the vector F_{s_i} and F_{s_j} , which is represented as $corr(s_i, s_j)$. Moreover, closely stations tend to have similar patterns and are more likely to belong to the same group. So we combined the distance factor into the similarity calculation, and give the distance and bike usage pattern similarity (DAPSimi) as follow:

$$DAPSimi(s_i, s_j) = \frac{corr(s_i, s_j)}{Dist(s_i, s_j)} \times sign(\delta - Dist(s_i, s_j)), \quad (4)$$

where $sign(x)$ is a symbolic function.

4.1.3 Clustering methodology DBSCAN WINNER! See Why

After getting the central stations and the similarity between stations, we first define s_j with $DAPSimi(s_i, s_j)$ greater than a threshold SI as the neighbour of s_i . Secondly, we need to find the clustering core stations which satisfy the condition that it is a central station and the number of its neighbors is higher than a threshold M . There are many well-studied clustering algorithms like K-means [33] and DBSCAN [34], both of them can effectively deal with the problem of group similar stations. Compared DBSCAN with K-means, we found that DBSCAN can handle the clusters with any shapes and filter out the anomalies, while K-means treat all stations as normal. Moreover, the K-means algorithm needs to set the number of clusters before starting, which we can not get in advance. Thus, we choose the DBSCAN to generate the Station Clusters. This algorithm starts with any core stations and finds out the density-reachable stations that generate clusters until all core stations are visited. Then, the remaining stations in the δ -neighbourhood are added to the cluster with the most similar central station. Please note that we treat the stations with low similarity as outliers, and merged those clusters with fewer stations into their most similar neighbor cluster.

4.2 Service level calculation

Congestion == Jammed

In the dynamic BSS, the demand for bicycles varies with time and stations, resulting in congestion at some stations and starvation others. In order to capture the dynamics of the system and predict the real-time bicycle flow in the future, we first train a return and a rental predictor in this section and then designed a system simulator to simulate dynamic changes of BSS to determine the service level of each station.

4.2.1 Return and rental predictor

Since the potential bicycle demand is unknown, we estimate the actual return or rental demand of each station based on historical bicycle usage data and site status data. Combined with timestamps, weather, and other factors, we designed two predictors to predict the future bicycle rental or return demand. The equation of each predictor formalized as follows:

$$\lambda_s(t) = \frac{\sum_{n \in |N^+|} D_{s,\Delta t}^{n,+}}{|N^+| \times \Delta t}, \quad (5)$$

$$\mu_s(t) = \frac{\sum_{n \in |N^-|} D_{s,\Delta t}^{n,-}}{|N^-| \times \Delta t}, \quad (6)$$

where N^+ and N^- are days with the most similar historical time slot with Δt under the same weather conditions. Only when the site s is not empty or full during the period Δt , can the bike usage record be selected to obtain the actual demand.

4.2.2 Service level of each station ?

Given the return and rental demand of each station at time t , we can simulate the flow of bikes between stations over a period in the future. Specifically, the number of bikes at station s after a time span Δt is estimated by:

$$x_{s,t+\Delta t} = x_{s,t} + \int_t^{t+\Delta t} (\lambda_s(t) - \mu_s(t)) dt. \quad (7)$$

Definition 4 (Service Level) The service level of a station s at time t is a set, in which store all the possible number of bikes or docks that can keep the station always available for customers to rent or return a bike for the future period T like 1 hour. We divide this period into n tiny windows, e.g. $(t, t + \Delta t], \dots, (t + (n-1)\Delta t, t + n\Delta t]$, and different time slots are mutually independent. Utilize the estimate given by (8), we define the service level of station s at time t as

$$SL(s, t) = \{x_{s,t} \mid x_{s,t} \in [1, C_s - 1] \text{ and } \forall k \in (0, n), x_{s,t+k\Delta t} \in [1, C_s - 1]\}. \quad (8)$$

Based on the service level, it is easy to determine whether a station will be unbalanced over the next period. We mark the station s as unbalanced if the number of real-time bikes is not in the service level range at the current time. And we can also calculate the minimum number of bikes that need to be transferred to keep the station in balance, which defined as

$$TN = \begin{cases} x_{s,t} - \max(SL(s, t)), & \text{if } x_{s,t} > \max(SL(s, t)), \\ \min(SL(s, t)) - x_{s,t}, & \text{if } x_{s,t} < \min(SL(s, t)), \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

This number can reflect how busy the station will be for the coming time period and can be utilized to calculate the Bikes Score (BS) later.

Definition 5 (Urgent Station) If station s is marked as an unbalanced station, and the real-time available bikes or docks less than or equal to 1 at time t , i.e. $x_{s,t} \leq 1$ or $C_s - x_{s,t} \leq 1$, we consider this station s as an urgent station.

4.3 Decision generation

When repositioning bikes, there are many stations that the truck can choose to visit in the next moment, but it is uncertain which one is the best choice. Due to the powerful decision ability of MCTS, we introduced it to eliminate the uncertainty in the station selection process.

MCTS rests on two fundamental concepts [35]: (1) the true value of an action can be approximated using random constant stimulation, and (2) these values may be used efficiently to adjust the strategy towards a best-first one. According to the state sequence sampled in each simulation process, we can construct a search tree and estimate the corresponding value of some possible actions in the current state based on this tree. In fact, In the initial stage of simulation, the model's estimation of the site value may have large errors, but as the number of exploration increases, the estimated reward value will become more accurate.

In our problem, nodes in the search tree represent stations, and the edges denote the possible actions that correspond to the station. Due to the dynamic nature of BSS, we only generate the values for all possible actions at the current station, rather than making a series of decisions in advance. Specifically, when deciding which station to visit next, we first perform the simulate process and estimate the reward value for possible stations. Then we choose the station with the maximum value as the next target.

4.3.1 Station reward

To indicate the priority of different unbalanced stations, we need to design a function to evaluate the value of each

Table 2 Distances score

Travelled distance	DS
0km - 0.3km	10
0.3km - 0.6km	6
0.6km - 1.0km	4
1.0km - 1.5km	2
1.5km - 2.0km	0
2.0km -	-5

station. We denote the station visited at round r as A_r , and defined the corresponding reward value as

$$Q = \alpha \times DS + (1 - \alpha) \times BS + US + Q_f, \quad (10)$$

where DS , BS and US are the scores shown in Tables 2, 3 and 4, which are set by experience, α controls the importance of distance and bikes' number as a trade-off. Q_f is the average reward value of the subsequent unbalanced stations, which represents the impact of the subsequent unbalanced stations in δ - neighbourhood of A_r .

4.3.2 UCT for exploration and exploitation

Multi-armed Bandit (MAB) [36] is a well-known sequential decision problem. Given a machine with several rocker arms and each arm has different rewards, the ultimate goal of users is to maximize the total rewards gained by choosing the optimal arms. It is difficult to choose an arm as the underlying reward distributions for each arm are unknown, and potential rewards must be estimated based on past observations. Thus it leads to the exploitation-exploration dilemma: should one exploit the action currently believed to be optimal or explore other actions that currently appear suboptimal but may turn out to be superior in the long run?

We model the process of determining the next station in each reposition round as an independent MAB problem. Thus, the exploitation-exploration dilemma is inevitable

Table 3 Bikes score

Transferred number	BS
0 bike	0
1 bike - 3 bikes	2
4 bikes - 6 bikes	4
7 bikes - 10 bikes	6
11 bikes -	10

Table 4 Urgency score

Station status	US
Urgent Station	10
other station status	0

in our model. In the initial exploration stage, the model explores the environment less and lacks useful prior information, so the random strategy is mainly adopted for station selection. As the number of simulations increases, the truck tends to choose the station with the highest value currently based on previous experiences. However, whether a station is indeed the optimal one can only be ascertained after explored a sufficient number of times for all other stations. To deal with the exploitation-exploration dilemma, we employed the Upper Confidence Bound for Trees (UCT) algorithm [37] in this paper. In round r , a child node A_r is selected to maximize UCT:

$$A_r = \arg \max_{a \in S} (Q_{r-1}(a) + \sqrt{\frac{c \ln N}{N_{r-1}(a)}}), \quad (11)$$

where $Q_{r-1}(a)$ is the expected reward value of node a in the first $r - 1$ rounds, N represents the number of times the current (parent) node has been visited, $N_{r-1}(a)$ denotes the number of times node a has been visited in the first $r - 1$ rounds. The variable c is a positive weight, which controls the trade-off between exploitation and exploration.

The two terms in (12) represent exploitation and exploration separately. In the initial stage, stations have not been explored adequately, and the UCT value is mainly dominated by the exploration term. Thus, even a node with a low expected reward value can also guarantee to be explored. With the increase of exploration times, the proportion of exploration value in the final reward value gradually decreases. For nodes with low expected reward values, the frequency of selection decreases as the rounds increases. In this way, The UCT algorithm ensures that all nodes are explored adequately and avoids a local optimum. The following Example 1 will show the process mentioned above.

Example 1 (The process of MCDR simulation) In Fig. 2, circles represent stations (the child nodes of each node represent the adjacent stations of the parent node), arrows between nodes represent the paths between them, and the truck stops at station s_1 . When beginning to plan the route

for the truck, we need to calculate the Q value for each station. In the first few rounds, we can find that Q_{s_2} calculated by the (10) may be greater than Q_{s_6} since it just finds the length of route $\{s_1 \rightarrow s_2\}$ is shorter than $\{s_1 \rightarrow s_6\}$. Nevertheless, the Q value for each station can not approximate the real Q value at this time due to inadequate exploration. And we can see from Fig. 2 that compared with s_2 , there are more unbalanced stations in the subsequent all possible sub-stations set of s_6 . Therefore, after sufficient exploration of all stations and potential optimal routes, the value of Q_{s_2} will gradually decrease, and Q_{s_6} will increase. Eventually, according to the Q value calculated before, the truck will reposition bikes towards the direction of station s_6 .

Algorithm 1 MCDR algorithm.

Input: the available stations in one cluster $availS$, the current location $rootS$ where the truck starts to reposition bikes;

Output: s^* ;

```

1: for  $s_i$  in  $availS$  do
2:    $Q(s_i) = -\infty$ ;
3:    $N(s_i) = 0$ ;
4: end for
5:  $r = 0$ ;
6:  $t = CurrentTime()$ ;
7: repeat
8:    $availSCopy = copy(availS)$ ;
9:    $Q = SIMULATION(availSCopy, rootS, N, r, t)$ ;
10:   $r = r + 1$ ;
11: until the  $Q$  of convergence
12: get unbalanced stations set  $USS$  by  $SL$  at time  $t$ ;
13:  $AUSS = availS \cap USS$ ;
14:  $s^* = \arg \max_{s_i \in AUSS} Q(s_i)$ ;
15: remove  $s^*$  from  $availS$ ;
16: return  $s^*$ ;
```

4.3.3 MCDR for dynamic repositioning

We illustrated the detailed procedure of our MCDR model in Algorithm 1. First, the program initializes the reward value and visit times for each station before the training process (lines 1-3). Then, the simulate process of the MCDR model is executed (lines 4-11). By continuously simulating and updating the Q value from the current location, we can get the final Q value for each station. In lines 12-14, the unbalanced station s^* with the largest Q value is selected as the next target station to visit. We removed s^* from $availS$ after s^* is visited (line 15).

Algorithm 2 Simulation algorithm.

Function *SIMULATION* (*availS*, *rootS*, *N*, *r*, *t*)

```

1: path = {}, dis = 0.
2: repeat
3:   if  $\forall s_i \in \text{availS}, N(s_i) > 0$  then
4:      $A_r = \arg \max_{a \in \text{availS}} (Q_{r-1}(a) + \sqrt{\frac{c \ln(r-1)}{N(a)}})$ .
5:   else
6:     if  $\text{random}(0, 1) < \varepsilon$  then
7:        $A_r = \text{choice}(\{\text{nei}(\text{rootS})\} \cap \{\text{availS}\})$ .
8:     else
9:        $A_r = \text{choice}(\text{availS})$ .
10:    end if
11:  end if
12:  remove  $A_r$  from availS.
13:   $N(A_r) += 1$ .
14:   $\text{path} = \text{path} \cup A_r$ .
15:   $\text{dis} += \text{Dis}(\text{rootS}, A_r)$ .
16:   $\text{rootS} = A_r$ .
17:  if  $A_r$  is unbalanced station at time t then
18:    get TN, US, Qf.
19:     $Q(A_r) = \alpha \times DS + (1-\alpha) \times BS + US + Q_f$ .
20:    for each s in path do
21:       $Q(s) = Q(s) + (Q(A_r) - Q(s))/N(s)$ .
22:    end for
23:    update time t.
24:    break.
25:  end if
26:  update time t.
27: until availS is null.
28: return Q.
```

Every time the truck finishes the current reposition and prepares to choose another station to visit, the simulation process will begin to execute. If there are few or no bikes in the truck, the truck will choose congested stations to pick up bikes. Similarly, a full or nearly full truck always chooses empty or nearly empty stations to drop off bikes. We displayed the simulation function in Algorithm 2. Note that before using the (12), we should make sure that all stations have been visited at least once. With a probability ε , A_r is selected from the available neighbors of the current station. Otherwise, A_r is replaced by an available station in *availS*. Once A_r is an unbalanced station, we should update the reward value *Q* in this simulation (lines 17-25). Provided that A_r is not an unbalanced station, the simulation process continues until an unbalanced station is found or all available stations in *availS* are visited. Here is a special situation we need to declare. If the remaining number of bikes or docks at a station still cannot meet its service level after the repositioning is completed, we still treat it as an unbalanced station and keep it in the candidate set (*availS*).

We estimate the completion time for one repositioning process in the simulation as follows:

$$t_{s_j, \text{end}} = t_{s_i, \text{end}} + \frac{\text{dis}_{s_i, s_j}}{v_{\text{truck}}} + TN \times \gamma + \phi, \quad (12)$$

where $t_{s_i, \text{end}}$ and $t_{s_j, \text{end}}$ represent the time for finishing repositioning bikes at station s_i and s_j separately, dis_{s_i, s_j} is the Manhattan distance between s_i and s_j , v_{truck} is the speed of the truck. The variable γ denotes the constant time for picking up or dropping off a bike, and ϕ is a reposition noise.

The time complexity of our MCDR model to get the next choice for a truck is $st \cdot O(\text{availS})$, where *st* is the simulation times and *availS* is the available stations. And if an unbalanced station is selected during one simulation, the time complexity will be $st \cdot O(\text{availS}) \cdot O(n)$, where *n* is the number of stations visited in one simulation.

5 Experiments

In this section, we evaluate our method based on a real dataset from New York Citi Bike system¹ and discuss the experimental results of morning and evening rush hours in two clusters. All experiments are conducted on a PC with an Intel(R) Core i5-7500 CPU, 3.4GHz, and 12 GB RAM running a 64-bit Windows 10 system. Some experimental parameters used in this paper summarized in Table 5. The value of the parameters τ , *SI* and *M* are determined by Grid Search method [38] with the ranges of them in 1.1-2.0, 1.0-2.0 and 5-10 respectively. We follow the setting of parameter *T* in [3, 25] and set it as 1 hour. It is worth noting that we only consider using one kind of truck for bike repositioning, and assume that all workers have the same work efficiency. Therefore, we set the parameters related to vehicles and workers (i.e. C_{truck} , v_{truck} , and γ) in the table as fixed values. In order to simulate the uncertainty in the real environment, we also employed a normal distribution with the mean value of 0 and variance of 2 to introduce the reposition noise. Moreover, we further studied the influence of some parameters in Section 5.3.3.

5.1 Real-world datasets

We adopted three real-world datasets in our experiments, i.e. the Citi Bike usage data, the weather data, and the station status data in New York City². The Citi Bike usage data we used spans ten months from May 2017(2018) to October 2017(2018), consisting of trip duration, start station ID, end station ID, start time, and end time. To obtain a

¹<http://www.capitalbikeshare.com/system-data>.

²<https://github.com/liao626/stationStatusData>

Table 5 Parameters used in this paper

Parameters	Values
Centrality threshold τ	1.4
Similarity threshold SI	1.7
Core site threshold M	8
Time span Δt	5 min
Future period T	1 hour
Truck capacity C_{truck}	40
Truck speed v_{truck}	25 km/h
Pick/Drop time γ	0.3 min
Reposition noise ϕ	$N(0, 2)$ min
Neighbor distance threshold δ	0.75 km
Selection probability ε	0.7
Positive weight c	10

more accurate prediction of demand, we also collected the weather conditions from a public meteorological website³. The station status data includes time, station ID, available bikes, available docks, total docks, and status value, which is necessary to calculate the service level in different periods. The station status is recorded every 5 minutes for each station, and we removed those records with the ‘not in service’ state.

5.2 Baselines and evaluation metric

We compare the performance of our method with several baselines list below:

No Repositioning (NR) We simulate the bike flows without using and repositioning strategy in this method and calculate the customer loss.

Static Repositioning (SR) [10] SR minimizes the routing costs for each vehicle based on an optimization model. This model does not consider the dynamics of BSS, but only allocates a fixed number of bicycles to each station at the beginning or end of the day according to the historical demand. Since the initial number of bikes at the station is almost the same for each weekday, we compare this model only in terms of the customer loss reduction ratio.

Nearest Neighbor Repositioning (NNR) NNR is a greed-based dynamic bike reposition model which prioritizes stations that are the closest or most unbalanced. This method set the number of bikes or docks to be a constant instead of predicting it dynamically

Nearest Neighbor Repositioning based Service Level (NNRSL) A variant of NNR that predicts the future bike demand for stations using the service level mentioned in Section 4.2.

Spatio-Temporal Reinforcement Learning Approach (STRL)

[20] The STRL model sets the negative of customer loss after repositioning bikes as the reward and applies the Bellman equation to find the optimal strategy. Nevertheless, **the effect of the distance traveled on the reward value is not taken into account in this model**. Generally, the longer the travel distance of trucks, the higher the cost and the more serious the customer loss. **We performed this experiment under the same dataset and parameters as MCDR. And we set the dimension of the 1st and the 2nd FC layer to 64 and 32.**

Gap!
article
10

Evaluation metric To evaluate the performance of our method, we first obtain the repositioning result (total customer loss) of each model and then introduce the concept of

the loss reduction ratio (lrr) as the final metric, which is defined as:

$$lrr = \frac{L - L_m}{L}, \quad (13)$$

where L is the total loss of customers in one cluster with no bike repositioning, and L_m denotes the total loss of customers under a specific repositioning strategy mentioned in baselines.

5.3 Results

Experiment results of our method are shown and detailly analyzed in this section. The parameters used in our experiments are given in Table 5.

5.3.1 Clustering results

We conducted the clustering algorithm in a rectangle area of New York City and shown the results in Fig. 4. After merging the small clusters, the 570 stations in BSS are grouped into 30 clusters (different colors in the figure), with 44 stations in the largest and 16 stations in the smallest. Here we show two representative clusters C_1 and C_2 in Fig. 4b and c and detail the information of each cluster in Table 6. In this table, p_1 and p_3 denote the time period of the morning and evening rush hours respectively, $D_{p_1}^-$ and $D_{p_1}^+$ is the corresponding average rental and return demand.

³<https://www.wunderground.com/weather/api/>

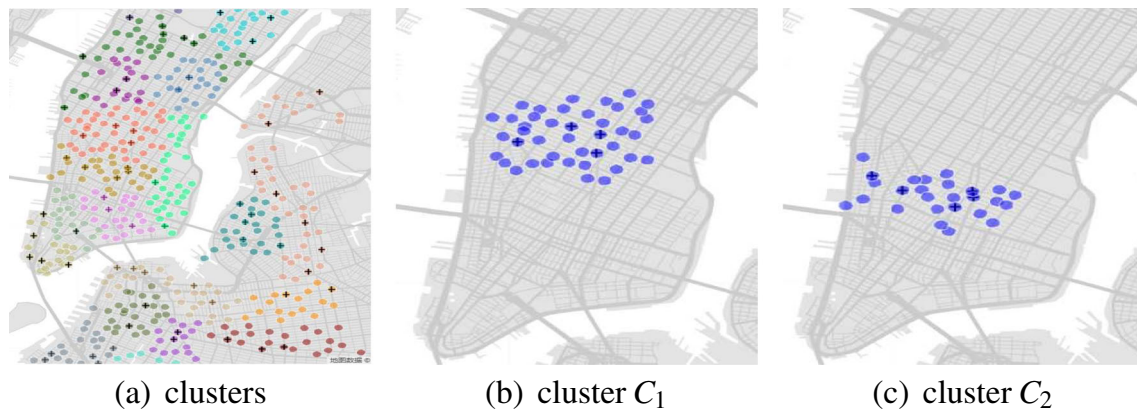


Fig. 4 The result of stations clustering, where the circles with different colours denote stations in different groups. C_1 and C_2 are the clusters that used in our experiments

5.3.2 Method comparison

We compared our method with other baselines list in Section 5.2 and displayed the trend of customer loss over weekdays of a month under different approaches in Fig. 5. The results show that without using bike repositioning, the customer loss will reach an intolerable level, and the curve changes dramatically. Although SR and NNR can slightly reduce customer loss, neither of them performs stably without considering the future uncertainties and status of stations. NNRS� shows a great advantage against NNR because it predicts the future demand of stations based on the service level instead set a constant as done in NNR. However, NNRS� is prone to fall into a local optimum without considering long-term repositioning. STRL and MCDR have similar loss curves, and both of them can effectively deal with the imbalance of bikes and keep the customer loss in a stable and acceptable range. In our method, multiple factors are taken into account, e.g. the station state, the distance between trucks and stations, and the demand of each station. Moreover, we defined a reward value Q to evaluate the priority of each station and train a long-term strategy to avoid local optimization. So there is no accident that our method outperformance all baselines above.

Based on the total loss without bike relocation strategies and the real loss we obtained by using all the methods in this paper, we further studied the lrr metric and shown the results in Fig. 6. The experimental results demonstrate the strength of our method in terms of the metric lrr . There is no doubt that MCDR can surpass those two static repositioning methods (i.e. SR and NNR). Although NNRS� made a huge improvement by predicting the future demand, MCDR still shows great superiority because that NNRS� is prone to fall into a local optimum without considering long-term repositioning. Specifically, MCDR achieves a better performance than STRL by 0.146 in p_1 and 0.107 in p_3

respectively for C_1 , 0.086 and 0.072 for C_2 . The reason is that STRL ignores the distance between trucks and stations, and it only focuses on customer loss after bike repositioning. And we noticed that NNR got a worse performance than SR in cluster C_2 . We think that was because the stations in cluster C_1 are closer than those in C_2 , which makes the greedy algorithm focus more on the nearest stations instead of those with more unbalanced neighbors.

To illustrate how the reward value Q of each station changes when training MCDR in the simulations. We randomly choose several stations in cluster C_1 and C_2 and draw the variation curves of their Q value with the number of iterates in Fig. 7. We can see that at the beginning of the iteration, all stations have a fluctuating Q value, but with the increase of the number of iterations, they begin to stabilize and finally reach a stable state. The reason is that in the initial stage of MCDR model training, only a few stations in the BSS are visited, and the value of UCT in (11) mainly depends on the number of visited stations. As the iterations increase, the Q values of stations are updated iteratively and frequently. When all stations have been explored a considerable number of times, the UCT

Table 6 The detailed information of clusters

Cluster	C_1	C_2
stations	42	26
central stations	4	5
max distance between stations	3.083km	3.033km
min distance between stations	0.105km	0.121km
average distance	1.291km	1.134km
$D_{p_1}^-$ for each weekday of a month	1853	1171
$D_{p_1}^+$ for each weekday of a month	2098	1079
$D_{p_3}^-$ for each weekday of a month	2706	1661
$D_{p_3}^+$ for each weekday of a month	2575	1856

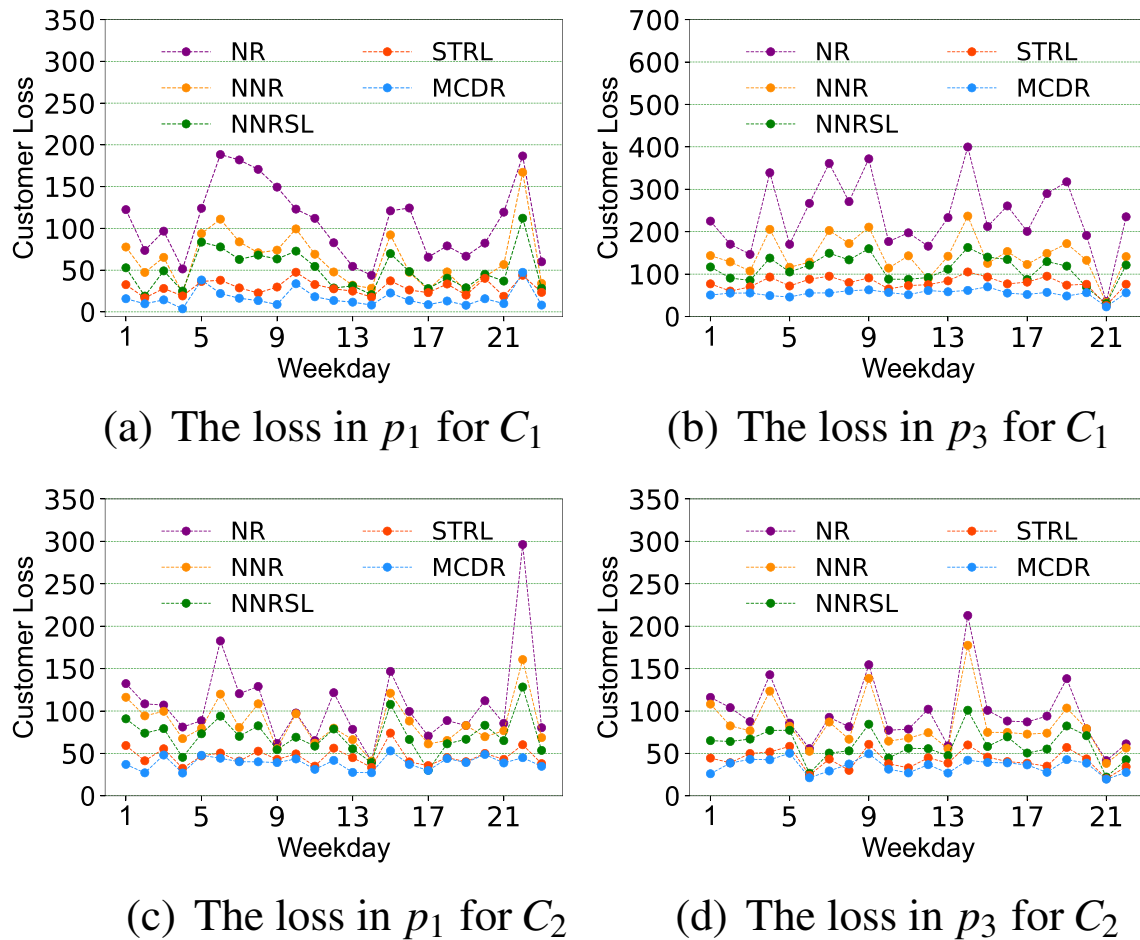


Fig. 5 The loss of customers

value will be determined by the size of the Q_{r-1} value, and the Q value of the stations tend to be stable.

5.3.3 Influence of parameters

We further study the influence of our model parameters in this section, and the result display as follows.

Different Discount Factor As shown in Fig. 8, we discussed the impact of different α in the (11) on the customer loss and the distance travelled in cluster C_1 . It is obvious that with the increase of α , the distance that trucks need to travel becomes shorter, while the customer loss gradually increases. The reason is that our model employs α to tradeoff the impact of the distance and bike usage over the reward value Q . When

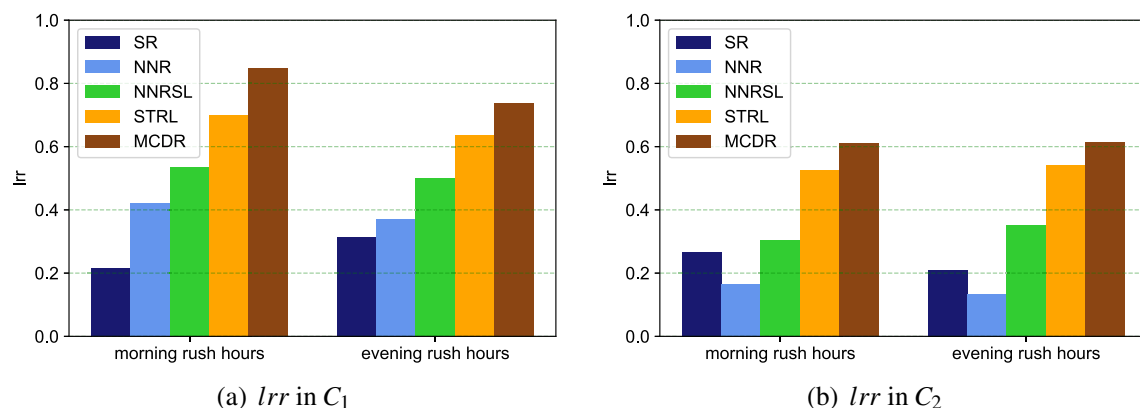


Fig. 6 Customer loss reduction ratio

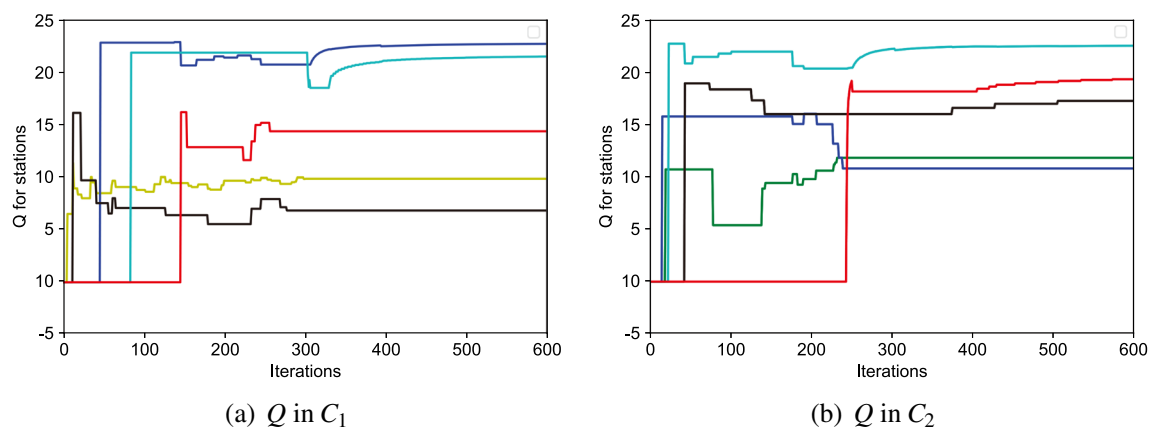
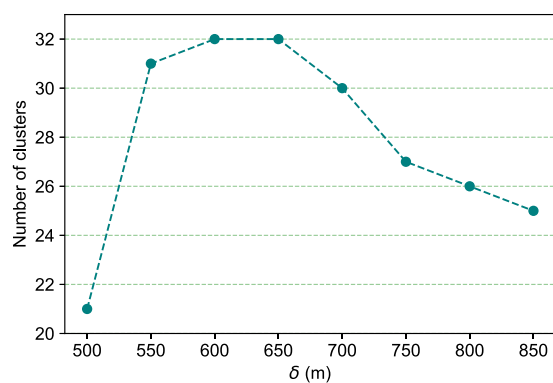
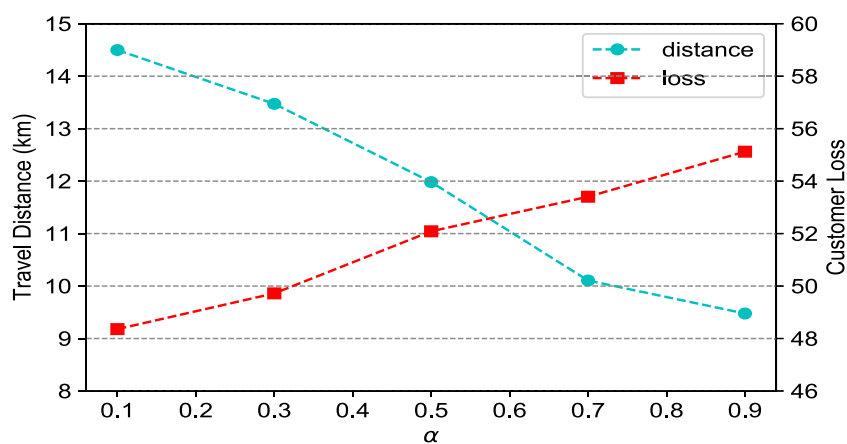
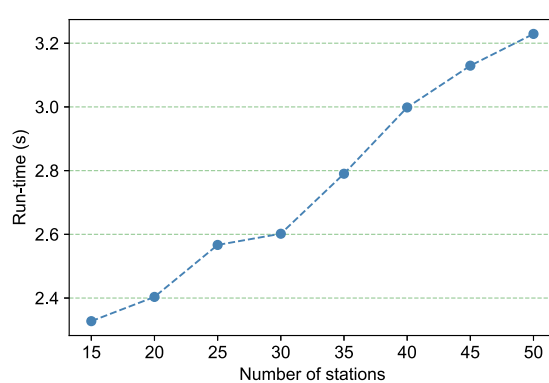


Fig. 7 The convergence of Q , where each line denotes the change in Q value of a station over the number of iterations

Fig. 8 Impacts of α



(a) Impacts of δ



(b) Impacts of cluster size

Fig. 9 Impacts of the neighborhood threshold

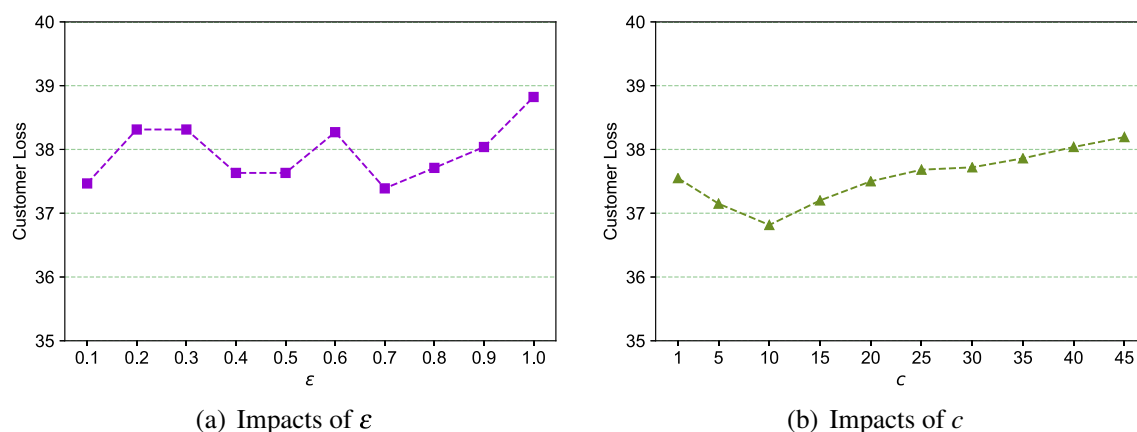


Fig. 10 Parameters impact analysis on the selection probability and positive weight

the α is small, our model considers more about the real-time usage of bikes regardless of travel distance. And if we set a larger α , the reward function will encourage the trucks to visit the nearest stations, which can not utilize the real-time supply and demand information and leads to myopic strategies.

Different Neighbourhood Threshold We also plot the impact of the neighborhood threshold δ on the clustering performance in Fig. 9a. Since a small neighborhood threshold, δ may filter more stations with low similarity *DAPSimi* between pairwise stations, but a very large δ will merge some adjacent groups, which causes the number of clusters first increases and then decreases after $\delta = 650m$. Figure 9b show the running time of MCDR changing over the sizes of clusters. As the number of stations in a cluster increases, the search space becomes larger, and the MCDR model needs to spend more time to learn the optimal station selection. In general, smaller clusters can achieve better performance but need more trucks and employees. To guarantee the efficacy and keep the cost reasonable, we choose 750m as the neighborhood threshold δ in our experiments.

Different Selection Probability and Positive Weight Figure 10a displays the impact of selection probability ϵ on our method. The curve of the customer loss fluctuates very little with the ϵ value increases, and the minimum value is obtained at $\epsilon = 0.7$. For parameter c , we evaluate the impact of it on the customer loss in the range of 1-40. A small c makes our method focus on those stations with larger reward value, but a very large c may ignore the valuable knowledge obtained before. That is why the customer loss decreases and then increases after a threshold like $c = 10$ in Fig. 10b. In Section 5, we mainly choose $\epsilon = 0.7$ and $c = 10$ to conduct our experiments.

6 Conclusions

In this paper, we addressed the bike stock imbalance problem in BSS and developed a novel bike repositioning method named MCDR to tackle this issue. Focusing on improving the availability of the system and reduce the loss of customers, the proposed method first divides the whole BSS into multiple groups, and estimate the demand of stations in each group, and finally generates an optimal (minimum user loss) path for trucks to relocate bikes between stations. The experimental results based on a real dataset indicate that our method outperforms the baselines in terms of the *lrr* score and the stability. Our method gets an average *lrr* score around 0.7, which outperforms all the baseline methods. Besides, the experimental results in different regions also show the superiority of the proposed method in terms of stability. In future work, we plan to study the optimal number of trucks in each cluster and try to take the temporary truck scheduling (bike repositioning) strategy between adjacent groups into consideration.

Acknowledgements The work was supported in part by the National Science Foundation of China grants 61876138. Any opinions, findings, and conclusions expressed here are those of the authors and do not necessarily reflect the views of the funding agencies.

Declarations

Conflict of Interests The authors declare that they have no conflict of interest.

References

- Meddin R, DeMaio P (2018) The bike-sharing world map. <http://www.bikesharingworld.com>
- Gao X, Lee GM (2019) Moment-based rental prediction for bicycle-sharing transportation systems using a hybrid genetic

- algorithm and machine learning. *Comput Ind Eng* 128(December 2018):60–69. <https://doi.org/10.1016/j.cie.2018.12.023>
3. Hulot P, Aloise D, Jena SD (2018) Towards station-level demand prediction for effective rebalancing in bike-sharing systems. In: Guo Y, Farooq F (eds) *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19–23, 2018*. ACM, pp 378–386
 4. Sathishkumar V, Park J, Cho Y (2020) Using data mining techniques for bike sharing demand prediction in metropolitan city. *Comput Commun* 153(January):353–366. <https://doi.org/10.1016/j.comcom.2020.02.007>
 5. Cagliero L, Cerquittelli T, Chiusano S, Garza P, Xiao X (2017) Predicting critical conditions in bicycle sharing systems. *Computing* 99(1):39–57. <https://doi.org/10.1007/s00607-016-0505-x>
 6. Zhou Y, Li Y, Zhu Q, Chen F, Shao J, Luo Y, Zhang Y, Zhang P, Yang W (2019) A reliable traffic prediction approach for bike-sharing system by exploiting rich information with temporal link prediction strategy. *Transactions in GIS* 23(5):1125–1151. <https://doi.org/10.1111/tgis.12560>
 7. Jia W, Tan Y, Liu L, Li J, Zhang H, Zhao K (2019) Hierarchical prediction based on two-level Gaussian mixture model clustering for bike-sharing system. *Knowl-Based Syst* 178:84–97. <https://doi.org/10.1016/j.knosys.2019.04.020>
 8. Li Y, Zheng Y (2020) Citywide bike usage prediction in a bike-sharing system. *IEEE Trans Knowl Data Eng* 32(6):1079–1091. <https://doi.org/10.1109/TKDE.2019.2898831>
 9. Raviv T, Tzur M, Forma IA (2013) Static repositioning in a bike-sharing system: models and solution approaches. *EURO J. Transport. Logist.* 2(3):187–229. <https://doi.org/10.1007/s13676-012-0017-6>
 10. Schuijbroek J, Hampshire RC, van Hoes WJ (2017) Inventory rebalancing and vehicle routing in bike sharing systems. *Eur J Oper Res* 257(3):992–1004. <https://doi.org/10.1016/j.ejor.2016.08.029>
 11. Wang Y, Szeto WY (2018) Static green repositioning in bike sharing systems with broken bikes. *Transport. Res. Part D Transp. Environ.* 65(September):438–457. <https://doi.org/10.1016/j.trd.2018.09.016>
 12. Vogel P, Greiser T, Mattfeld DC (2011) Understanding bike-sharing systems using Data Mining: Exploring activity patterns. *Procedia. Soc. Behav. Sci.* 20:514–523. <https://doi.org/10.1016/j.sbspro.2011.08.058>
 13. Ghosh S, Varakantham P, Adulyasak Y, Jaillet P (2017) Dynamic repositioning to reduce lost demand in bike sharing systems. *J Artif Intell Res* 58:387–430
 14. Ghosh S, Varakantham P, Adulyasak Y, Jaillet P (2015) Dynamic redeployment to counter congestion or starvation in vehicle sharing systems. In: *Proceedings of the 25th International Conference on Automated Planning and Scheduling, ICAPS 2015, Jerusalem, Israel, June, 7–11, 2015*, pp 79–87
 15. Lowalekar M, Varakantham P, Ghosh S, Jena SD, Jaillet P (2017) Online repositioning in bike sharing systems. In: *Barbulescu L, Frank J, Mausam SSF (eds) Proceedings of the Twenty-Seventh International Conference on Automated Planning and Scheduling, ICAPS 2017, Pittsburgh, Pennsylvania, USA, June 18–23 2017*, pp 200–208
 16. Ghosh S, Trick M, Varakantham P (2016) Robust repositioning to counter unpredictable demand in bike sharing systems. In: *Kamshampati S (ed) Proceedings of the 25th International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9–15 July 2016*, pp 3096–3102
 17. Ghosh S, Koh JY, Jaillet P (2019) Improving customer satisfaction in bike sharing systems through dynamic repositioning. In: *Kraus S (ed) Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10–16, 2019*, ijcai.org, pp 5864–5870. <https://doi.org/10.24963/ijcai.2019/813>
 18. Qin R, Kong L, Guo M, Yao B, Guizani M (2018) Rebalance Modern Bike Sharing System: Spatio-Temporal Data Prediction and Path Planning for Multiple Carriers. In: *24th IEEE International Conference on Parallel and Distributed Systems, ICPADS 2018, Singapore, December 11–13, 2018*. IEEE, pp 1081–1086. <https://doi.org/10.1109/PADS.2018.8644999>
 19. Jia H, Miao H, Tian G, Zhou MC, Feng Y, Li Z, Li J (2020) Multiobjective bike repositioning in Bike-Sharing systems via a modified artificial bee colony algorithm. *IEEE Trans Autom Sci Eng* 17(2):909–920. <https://doi.org/10.1109/TASE.2019.2950964>
 20. Li Y, Zheng Y, Yang Q (2018) Dynamic bike reposition: a spatio-temporal reinforcement learning approach. In: *Guo Y, Farooq F (eds) Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19–23, 2018*. ACM, pp 1724–1733. <https://doi.org/10.1145/3219819.3220110>
 21. Pfrommer J, Warrington J, Schildbach G, Morari M (2014) Dynamic vehicle redistribution and online price incentives in shared mobility systems. *IEEE Trans Intell Transport Syst* 15(4):1567–1578. <https://doi.org/10.1109/TITS.2014.2303986>. arXiv:1304.3949
 22. Singla A, Santoni M, Bartók G, Mukerji P, Meenen M, Krause A (2015) Incentivizing users for balancing bike sharing systems. In: *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25–30, 2015, Austin, Texas, USA*. AAAI Press, pp 723–729. <http://www.aaai.org/ocs/index.php/AAAI/AAAI15/paper/view/9942>
 23. Ghosh S, Varakantham P (2017) Incentivizing the use of bike trailers for dynamic repositioning in bike sharing systems. In: *Barbulescu L, Frank J, Mausam SSF (eds) Proceedings of the Twenty-Seventh International Conference on Automated Planning and Scheduling, ICAPS 2017, Pittsburgh, Pennsylvania, USA, June 18–23 2017*, pp 373–381
 24. Angelopoulos A, Gavalas D, Konstantopoulos C, Kypriadiis D, Pantziou G (2018) Incentivized vehicle relocation in vehicle sharing systems. *Transport Res Part C: Emerg Technol* 97(October):175–193. <https://doi.org/10.1016/j.trc.2018.10.016>
 25. Yi P, Huang F, Peng J (2019) A rebalancing strategy for the imbalance problem in bike-sharing systems. *Energies* 12(13). <https://doi.org/10.3390/en12132578>
 26. Ji Y, Jin X, Ma X, Zhang S (2020) How Does Dockless Bike-Sharing System Behave by Incentivizing Users to Participate in Rebalancing? *IEEE Access* 8:58889–58897. <https://doi.org/10.1109/ACCESS.2020.2982686>
 27. Chiariotti F, Pielli C, Zanella A, Zorzi M (2020) A bike-sharing optimization framework combining dynamic rebalancing and user incentives. *ACM Trans Auton Adaptive Syst (TAAS)* 14(3):1–30. <https://doi.org/10.1145/3376923>
 28. Garg N, Ranu S (2018) Route recommendations for idle taxi drivers: Find me the shortest route to a customer!. In: *Guo Y, Farooq F (eds) Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19–23 2018*, pp 1425–1434. <https://doi.org/10.1145/3219819.3220055>
 29. Huang F, Qiao S, Peng J, Guo B (2019a) A bimodal gaussian inhomogeneous poisson algorithm for bike number prediction in a bike-sharing system. *IEEE Trans Intell Transp Syst* 20(8):2848–2857. <https://doi.org/10.1109/TITS.2018.2868483>
 30. Huang J, Wang X, Sun H (2019b) Central station based demand prediction in a bike sharing system. In: *20th IEEE international conference on mobile data management, MDM 2019, hong kong, SAR, China, June 10–13 2019*, pp 346–348. <https://doi.org/10.1109/MDM.2019.00-38>

31. Liu J, Li Q, Qu M, Chen W, Yang J, Xiong H, Zhong H, Fu Y (2015) Station site optimization in bike sharing systems. In: Aggarwal CC, Zhou Z, Tuzhilin A, Xiong H, Wu X (eds) 2015 IEEE International conference on data mining, ICDM 2015, atlantic city, NJ, USA, November 14–17 2015, pp 883–888. <https://doi.org/10.1109/ICDM.2015.99>
32. Cohen J, Cohen P, West SG, Aiken LS (2013) Applied multiple regression/correlation analysis for the behavioral sciences. Routledge
33. MacQueen JB (1967) Some methods for classification and analysis of multivariate observations. In: In 5-th berkeley symposium on mathematical statistics and probability, pp 281–297
34. Birant D, Kut A (2007) ST-DBSCAN: An algorithm for clustering spatial-temporal data. Data Know Eng 60(1):208–221. <https://doi.org/10.1016/j.datak.2006.01.013>
35. Browne CB, Powley E, Whitehouse D, Lucas SM, Cowling PI, Rohlfshagen P, Tavener S, Perez D, Samothrakis S, Colton S (2012) A survey of monte carlo tree search methods. IEEE Trans Comput Intell AI Games 4(1):1–43. <https://doi.org/10.1109/TCIAIG.2012.2186810>
36. Sutton RS, Barto AG (1998) Introduction to reinforcement learning, vol 135. MIT Press, Cambridge
37. Kocsis L, Szepesvári C (2006) Bandit based monte-carlo planning. In: Fürnkranz J, Scheffer T, Spiliopoulou M (eds) European conference on machine learning (ECML). Springer, Berlin, pp 282–293. https://doi.org/10.1007/11871842_29
38. Lerman P (1980) Fitting segmented regression models by grid search. J R Stat Soc Series C (Appl Stat) 29(1):77–84. <https://doi.org/10.2307/2346413>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



He Li received his PhD degree from the Department of Information and Communication Engineering of Chungbuk National University, Rep. of Korea. He is now an Associate Professor in the School of Computer Science and Technology, Xidian University, China. His research interests include graph data management, data mining, and knowledge graph.



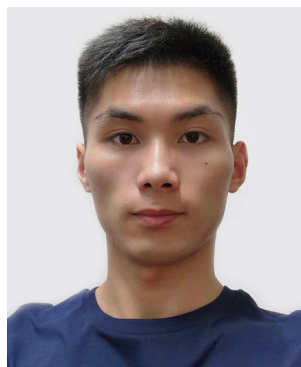
Ao Li is currently pursuing the master's degree with the School of Computer Science and Technology, Xidian University, Xi'an, China. His research interests include data mining, urban computing, and transport network modeling.



Jianbin Huang received the Ph.D. degree in pattern recognition and intelligent systems from Xidian University, Xi'an. In 2009, he held a postdoctoral research position with the University of Illinois at Urbana-Champaign. He is currently a Professor with the School of Computer Science and Technology, Xidian University. His research interests include data mining and knowledge discovery.



Longji Huang is a Ph. D. candidate at the School of Computer Science and Technology, Xidian University. His research interests include graph neural network, new network architecture designing, and the application of artificial intelligence related technologies in spatiotemporal trajectory data mining and spatiotemporal prediction.



Qinglin Tan is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Xidian University, Xi'an, China. His research interests include spatio-temporal data mining, reinforcement learning, and traffic pattern analysis.